

Como Analytics Engineer, tu responsabilidad no es solo entregar datos, sino asegurar la escalabilidad y la eficiencia de la plataforma.

En este sprint final, rediseñarás la arquitectura de los datos para optimizar costes (Partitioning), introducirás lógica analítica avanzada (Window Functions) y modelarás estructuras complejas (Arrays) para cerrar el ciclo con la automatización.

⚠ Importante

» Control de Costes: En BigQuery, cada consulta cuesta dinero. Antes de ejecutar nada, fíjate siempre en el mensaje del validador ("Dry Run") arriba a la derecha.

» Ubicación de los Datos: Asegúrate de que tu Dataset esté en la región EU (Multiple regiones) o las conexiones fallarán.

Limitaciones del Sandbox

La zona de pruebas de BigQuery (Sandbox) está sujeta a los siguientes límites:

» Se aplican todos los límites y cuotas habituales de BigQuery.

» Tienes 10 GB de almacenamiento activo y 1 TB de datos de consulta procesados cada mes sin coste alguno.

» ⚠ Caducidad: Todas las tablas, vistas y particiones caducan de forma automática después de 60 días.

» No admite funciones avanzadas como: Transmisión de datos (Streaming), DML (UPDATE/DELETE) o el Servicio de transferencia de datos.

Nivel 1: Entorno e Ingesta Híbrida (Code-First)

Escenario:

El cuadro de mando financiero tarda demasiado en cargar y el coste es inaceptablemente alto. Se ha identificado que el problema crítico son los informes que cruzan ventas con regiones geográficas.

Ejercicio 1:

Consulta sobre Tabla no Optimizada (Diagnóstico)

El Country Manager de Alemania necesita revisar urgentemente las transacciones del **día 12 de marzo de 2022**.

🔧 Tarea:

» 1. Escribe la consulta que une (JOIN) transacciones y compañías.

» 2. Filtra los resultados por la fecha indicada y el país "Germany".

» 3. Sin ejecutar la consulta, realiza un "Dry Run" (auditoría de costes).

📌 Observación: Fíjate que BigQuery lee casi toda la mesa a pesar de pedir solo un día (Full Table Scan).



Consulta sin título

Ejecutar

Guardar ▾

Programa

Archivo

Editar

Ejecutar

Herramientas

```
1 SELECT
2   t.*,
3   c.company_name,
4   c.country
5 FROM
6   `sprint3-analytics-albert.sprint3_silver.transactions_clean` AS t
7 JOIN
8   `sprint3-analytics-albert.sprint3_silver.companies_clean` AS c
9 ON
10  t.business_id = c.company_id
11 WHERE
12   DATE(t.timestamp) = '2022-03-12'
13  AND c.country = 'Germany';
```



Esta consulta procesará 11.86 MB cuando se ejecute.

< Información del trabajo		Resultados	Visualización
ID de trabajo	sprint3-analytics-albert:EU.bquxjob_479409dd_19ec1fd4fad (Ver detalles de rendimiento)		
Usuario	a.armentano73@gmail.com		
Ubicación	EU		
Hora de creación	13 jun 2026, 7:17:44 p.m. UTC+2		
Hora de inicio	13 jun 2026, 7:17:44 p.m. UTC+2		
Hora de finalización	13 jun 2026, 7:17:45 p.m. UTC+2		
Duración	0 s		
Bytes procesados	11.86 MB		
Bytes facturados	20 MB		
Milisegundos de ranura	178		
Prioridad del trabajo	INTERACTIVE		
Usar SQL heredado	false		
Tabla de destino	Tabla temporal		

DESARROLLO:

- **DATASET** : SILVER
- Consulta diagnóstico para unir tablas transaction_clean y companies_clean. Filtrado por fecha 12 de marzo 2022 y por Germany.
- **VALIDACIÓN** mediante DRY RUN : 11,86 MB
Descripción: El resultado muestra que, a pesar de solicitar solo un día concreto, BigQuery lee toda la tabla (**Full Table Scan**).

Ejercicio 2:

Re-arquitectura y Optimización del Almacenamiento (Partition & Cluster)
Escenario:

Tras el diagnóstico del ejercicio anterior, hemos confirmado que filtrar la tabla de transacciones por fecha o por empresa provoca una Full Table Scan (lectura completa de la tabla), lo cual es ineficiente. Para solucionarlo en la capa de consumo (Gold), materializaremos una versión optimizada de la tabla de hechos.

Problema del sandbox: como utilizamos un entorno gratuito sin tarjeta de crédito, BigQuery borra automáticamente los datos de las particiones con más de 60 días de antigüedad. Dado que nuestros datos históricos son

antiguos, si particionamos directamente la tabla, ésta acabará quedando vacía. Para evitarlo, lo resolveremos en dos pasos: primero, actualizaremos automáticamente las fechas de los datos y, después, aplicaremos la optimización física.



Tarea:

Paso 1: Generación de Datos Recientes (Mocking Data)

Crea una tabla intermedia llamada `sprint3_silver.transactions_reciente` a partir de la tabla `sprint3_silver.transactions_clean`. Tu objetivo es mantener todas las columnas, pero sustituir el timestamp original por uno nuevo, generado aleatoriamente para que caiga dentro de los últimos 50 días.

» Pista Técnica: Para hacerlo sin escribir todas las columnas a mano, puedes utilizar **`SELECT * EXCEPT(timestamp)`**. Para calcular la fecha aleatoria, deberás restar una cantidad de días al instante actual. Te serán muy útiles las funciones **`TIMESTAMP_SUB()`**, **`CURRENT_TIMESTAMP()`**, y la combinación **`CAST(RAND() * 50 AS INT64)`** para generar los días de desfase.

Archivo Editar Ejecutar Herramientas

```
1 CREATE OR REPLACE TABLE `sprint3_silver.transactions_recent` AS
2 SELECT
3     *,
4     TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL CAST(RAND() * 50 AS INT64) DAY) AS timestamp
5 FROM
6     (
7     SELECT * EXCEPT(timestamp)
8     FROM `sprint3_silver.transactions_clean`
9     );
10
```

✓ Esta consulta procesará 11.09 MB cuando se ejecute.

< Información del trabajo		Resultados	Detalles de la ejecución
ID de trabajo	sprint3-analytics-albert:EU.bquxjob_230de1d4_19ec221e52a (Ver detalles de rendimiento)		
Usuario	a.armentano73@gmail.com		
Ubicación	EU		
Hora de creación	13 jun 2026, 7:57:42 p.m. UTC+2		
Hora de inicio	13 jun 2026, 7:57:42 p.m. UTC+2		
Hora de finalización	13 jun 2026, 7:57:44 p.m. UTC+2		
Duración	1 s		
Bytes procesados	11.09 MB		
Bytes facturados	12 MB		
Milisegundos de ranura	1685		
Prioridad del trabajo	INTERACTIVE		
Usar SQL heredado	false		
Tabla de destino	sprint3-analytics-		

DESARROLLO:

- Creamos una copia de la tabla `sprint3_silver.transactions_clean` y la nombramos `sprint3_silver.transactions_recent`.
- Conservamos todas las columnas igual pero con el campo `timestamp` reemplazado por uno aleatorio dentro de los últimos 50 días (lo que permite trabajar con datos simulados recientes y dentro del rango de expiración de 60 días).

Paso 2: Creación de la Mesa Optimizada (Partitioning & Clustering)

Ahora, crea tu tabla definitiva `sprint3_gold.fact_transactions_optimized` a partir de los datos recientes que acabas de generar en `transactions_recent`. Debes construir la sentencia DDL para configurar la tabla con las siguientes estrategias físicas de almacenamiento:

» Particionamiento (Partitioning): Divide la tabla por la fecha del campo `DATE(timestamp)`. Esto permitirá que las consultas que filtren por día (`WHERE date = ...`) sólo lean la partición necesaria.

» Clustering: Ordena los datos dentro de cada partición por business_id. Esto acelerará drásticamente los filtros por compañía y los cruces (JOINS) con la dimensión de empresas.

Archivo	Editar	Ejecutar	Herramientas
1	CREATE OR REPLACE TABLE	<u>`sprint3_gold.fact_transactions_optimized`</u>	
2	PARTITION BY DATE(timestamp)		
3	CLUSTER BY business_id		
4	AS		
5	SELECT *		
6	FROM	<u>`sprint3_silver.transactions_recent`</u> ;	
7			



 Esta consulta procesará 11.85 MB cuando se ejecute.

Resultados de...

ChatView all jobs

<Información del trabajoResultadosDetalles de la ejecu

ID de trabajo

sprint3-analytics-albert:EU.bquxjob_258a4aca_19ec2351d8a (Ver detalles de rendimiento)

Usuario

a.armentano73@gmail.com

Ubicación

EU

Hora de creación

13 jun 2026, 8:18:41 p.m. UTC+2

Hora de inicio

13 jun 2026, 8:18:42 p.m. UTC+2

Hora de finalización

13 jun 2026, 8:18:44 p.m. UTC+2

Duración

2 s

Bytes procesados

11.85 MB

Bytes facturados

12 MB

Milisegundos de ranura

17842

Prioridad del trabajo

INTERACTIVE

Usar SQL heredado

false

Tabla de destino

sprint3-analytics-

☆	fact_transacti...	🔍 Consulta
❗	Esta es una tabla particionada. Learn more	
<	Esquema	Detalles
	Vista previa	
Tipo de tabla	Particionada	
Particionada por	DAY	
Particionada en el campo	timestamp	
Vencimiento de la partición	60 días	
Filtro de partición	No es obligatorio	
Agrupada por	business_id	

DESARROLLO:

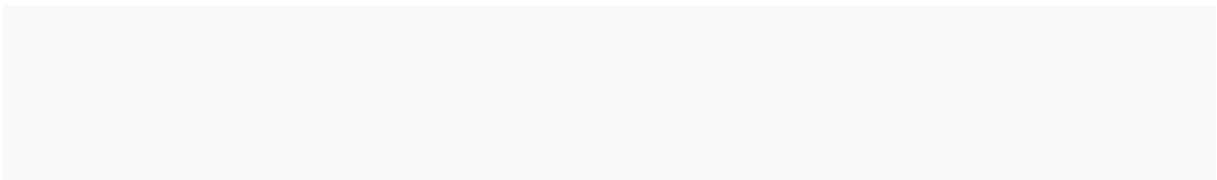
- Creamos una tabla definitiva (copia de la tabla ubicada en `sprint3_silver.transactions_recent`) en el Dataset `sprint3_gold` llamada `fact_transactions_optimized`.
- Aplicamos:
 - Partitioning: `DATA(timestamp)`: Divide la tabla por la fecha.
 - Clustering: `business_id` (dentro de cada partición los datos se ordenan por `business_id`). Agrupación.

CONCLUSIÓN:

- Mediante el Partitioning las consultas filtradas por día solo leen la partición necesaria y tienen menos bytes procesados. Y el Clustering permite optimizar los procesos de consulta ganando tiempo y rapidez.

 Entrega:

- » El código SQL utilizado para el Paso 1.
- » El código DDL (`CREATE OR REPLACE TABLE...`) utilizado en el Paso 2.
- » Una captura de pantalla de los "Detalles" de la nueva mesa `fact_transactions_optimized` donde se pueda comprobar que está particionada y agrupada en clústeres correctamente.



Ejercicio 3: La Prueba del Algodón (Benchmark)

Escenario:

Escenario: Ya has creado la tabla `sprint3_gold.fact_transactions_optimized` con las estrategias de Particionamiento y Clustering. Ahora toca demostrar al equipo la mejora de rendimiento y el ahorro de costes que has conseguido. Simularemos una petición típica de negocio: "Queremos ver todas las transacciones de los últimos 30 días".

 Tarea:

El objetivo es claro y directo: aplicar exactamente la misma consulta a dos tablas diferentes y comparar su coste computacional.

Construye una consulta SQL que seleccione todas las columnas (SELECT *) y filtre los datos de los últimos 30 días.

⚠ NO ejecutes las consultas todavía. En este ejercicio sólo utilizaremos el validador de BigQuery ("Dry Run", arriba a la derecha del editor) para visualizar los Bytes procesados de manera gratuita.

Haz la comparativa siguiendo estos dos pasos:

» Paso 1 (Tabla no optimizada): Apunta tu consulta a la mesa `sprint3_silver.transactions_reciente`. Observa el validador y haz una captura de pantalla de los bytes que procesaría.

» Paso 2 (Tabla optimizada): Sin cambiar nada de la lógica de tu consulta, modifica sólo el FROM para que ahora apunte a la mesa particionada `sprint3_gold.fact_transactions_optimized`. Observa cómo cambia el validador y haz la segunda captura.

👉 Entrega:

» El código SQL base que has diseñado.

» Las dos capturas de pantalla donde se vea la diferencia de "Bytes procesados" en el validador.

» Un breve texto calculando cuál es el % de ahorro que has conseguido (compara los bytes del Paso 1 respecto a los del Paso 2).

```
1 SELECT *
2 FROM `sprint3_silver.transactions_reciente`
3 WHERE DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY);
4
```

✅ Esta consulta procesará 11.85 MB cuando se ejecute.

```
1 SELECT *
2 FROM `sprint3_gold.fact_transactions_optimized`
3 WHERE DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY);
4
```

✅ Esta consulta procesará 7.23 MB cuando se ejecute.

CONCLUSIÓN:

- Hemos diseñado una consulta que selecciona todas las columnas de la tabla `sprint3_silver.transactions_reciente` y hemos aplicado exactamente la misma consulta sobre la tabla optimizada (particionada y clusterizada) `sprint3_gold.fact_transactions_optimized`. El indicador *Dry Run*, demuestra un ahorro del **38,96 %** en bytes procesados. Esto demuestra que las estrategias de particionamiento por fecha y clustering por `business_id` permiten reducir el volumen de datos leídos.

Ejercicio 4: Smart Caching (Vistas Materializadas)

Escenario:

El Director General tiene un cuadro de mando que muestra las "Ventas Totales por Día". Él (y 50 managers más) refrescan este gráfico constantemente.

Problema:

Si utilizas una consulta normal o una Vista estándar, cada vez que alguien refresca la página, BigQuery recalcula la suma de millones de filas. Es lento y estás pagando por el mismo cálculo una y otra vez.

Solución:

Utilizar una Vista Materializada. Esta tecnología guarda el resultado del cálculo en la memoria cae y sólo la actualiza si entran datos nuevos (incremental).

🧩 Tarea:

» Crea una Vista Materializada llamada `sprint3_gold.mv_daily_salas` que muestre las Ventas Totales por Día.

👉 **Entrega:**

» **Código para Crear la Vista Materializada.**

» Haz una consulta que muestre la vista creada y haz una captura de pantalla que muestre en su conjunto la vista materializada creada, con la consulta ejecutada y los bits procesados.

Archivo Editar Ejecutar Herramientas

```
1 CREATE MATERIALIZED VIEW `sprint3_gold.mv_daily_sales` AS
2 SELECT
3   DATE(timestamp) AS sale_date,
4   SUM(amount) AS total_sales
5 FROM `sprint3_gold.fact_transactions_optimized`
6 GROUP BY sale_date;
```

✓ Esta consulta procesará 0 B cuando se ejecute.

☆ mv_daily_sales 🔍 Consulta 💬 Chat Abrir en ▾

Esquema Detalles Estadísticas Linaje Perfil de datos

✦ Genera descripciones de tablas y columnas con Gemini. Aseg
perfil para [fundamentar](#) las descripciones.

Descartar Generar

Filtro Escribir el nombre o valor de la propiedad

☐	Nombre del campo	Tipo	Modo	Descripción
☐	sale_date	DATE	NULLABLE	-
☐	total_sales	FLOAT	NULLABLE	-

Archivo Editar Ejecutar Herramientas

```
1 SELECT *
2 FROM `sprint3_gold.mv_daily_sales`
3 ORDER BY sale_date DESC;
```

✓ Esta consulta procesará 816 B cuando se ejecute.

Resultados de...

ChatView all jobs

<

Información del trabajo

Resultados

Visualización

ID de trabajo

sprint3-analytics-albert:EU.bquxjob_2473552e_19ec54b0f46 ([Ver detalles de rendimiento](#))

Usuario

a.armentano73@gmail.com

Ubicación

EU

Hora de creación

14 jun 2026, 10:41:32 a.m. UTC+2

Hora de inicio

14 jun 2026, 10:41:32 a.m. UTC+2

Hora de finalización

14 jun 2026, 10:41:32 a.m. UTC+2

Duración

0 s

Bytes procesados

816 B

Bytes facturados

10 MB

Milisegundos de ranura

46

Prioridad del trabajo

INTERACTIVE

Usar SQL heredado

false

Tabla de destino

[Tabla temporal](#)

DESARROLLO:

- Utilizamos la Vista Materializada para:
 - Acelerar el rendimiento (BQ usa el resultado ya guardado y así evita el cálculo de millones de filas repetitivo).
 - Reduce costes de computación y se procesan menos bytes.
 - Mantiene los datos actualizados automaticamente. BQ recalcula las partes nuevas o modificadas. Los datos siempre estan al dia.
 - Puedes construir informes sin tocar la tabla original.
- BigQuery muestra 10 MB facturados por consulta (política estándar de facturación mínima), los Bytes procesado, 816 B son los que realmente indican el rendimiento.
- **CONCLUSIÓN:** la Vista Materializada del ejercicio obtenemos las **ventas totales por dia** con todas las ventajas antes comentadas.

Nivel 2: SQL Analítico Avanzado

Escenario:

La dirección empieza a hacer preguntas que no se resuelven con un simple GROUP BY. Necesitamos análisis de tendencias y comportamiento de usuario.

Ejercicio 1: Perfilado de Clientes VIP (Métricas Agregadas con CTEs)

Escenario:

El equipo de Marketing quiere analizar el comportamiento de nuestros mejores clientes para diseñar la estrategia del próximo año. Definimos "VIP" como aquellos con un gasto acumulado superior a 500€. Necesitan un informe que, para cada VIP, muestre su nombre, contacto y su patrón de compra: cuántas veces ha comprado, cuánto gasta de media y cuál fue su compra récord.

🔧 Tarea:

» 1. Crea una CTE llamada VIP_Stats que agrupe por usuario y calcule:

- El Gasto Total (SUM).
- La Cantidad de Transacciones (COUNT).
- El Ticket Medio (AVG), redondeado a 2 decimales.
- La Compra Máxima (MAX).
- Filtro: Mantienen sólo aquellos cuyo Gasto Total sea > 500.
- » Cruza la CTE con users_combined para obtener los datos personales.
- Requisitos de Salida:
- Columnas: user_id, nom_complet, email, num_compres, tiquet_mig, max_compra, total_gastat.
- Ordenado por total_gastat descendiente.

Archivo	Editar	Ejecutar	Herramientas
1	WITH VIP_Stats AS (		
2	SELECT		
3	user_id,		
4	SUM(amount) AS total_gastat,		
5	COUNT(*) AS num_compres,		
6	ROUND(AVG(amount), 2) AS tiquet_mig,		
7	MAX(amount) AS max_compra		
8	FROM `sprint3_gold.fact_transactions_optimized`		
9	GROUP BY user_id		
10	HAVING SUM(amount) > 500		
11	)		
12	SELECT		
13	v.user_id,		
14	CONCAT(u.name, ' ', u.surname) AS nom_complet,		
15	u.email,		
16	v.num_compres,		
17	v.tiquet_mig,		
18	v.max_compra,		
19	ROUND(v.total_gastat, 2) AS total_gastat		
20	FROM VIP_Stats v		
21	JOIN `sprint3_silver.users_combined` u		
22	ON v.user_id = u.user_id		
23	ORDER BY v.total_gastat DESC;		

Bytes procesados	1.79 MB
Bytes facturados	20 MB
Milisegundos de	252

DESARROLLO:

- **WITH VIP_Stats AS (** : Define la CTE(Common Table Expression) llamada VIP_Stats (es como una tabla temporal que solo existe dentro de esta consulta).
- **SELECT** : Selección de las columnas que formaran la CTE.
- **FROM**: indica la tabla origen donde estan las transacciones optimizadas
- **GROUP BY**: Agrupa todas las filas por usuario (user_id)
- **HAVING SUM(amount) > 500**: Filtra usuarios cuyo gasto es superior a 500.
- **)** : Cierra CTE.
- **SELECT (** : Seleccionamos las columnas definitivas que se verán en el resultado.
- **FROM VIP_Stats**: Tabla principal de la consulta (es la CTE).

- **JOIN:** Cruce de datos, conectamos las métricas con la Tabla users_combined.
- **ORDER BY:** Orden por total_gastat DESC.

Ejercicio 2: Análisis de Tendencias (Window Functions sobre Vistas)

Escenario:

La Dirección Financiera quiere monitorizar la "velocidad de ventas" diaria. Necesitan un informe que compare el rendimiento de cada día contra el día anterior para detectar caídas bruscas o picos de crecimiento (Day-over-Day Growth).



Estrategia de Optimización:

Para este cálculo NO debes ir a la tabla de hechos original ni recalcular sumas. Como buen Data Engineer, debes utilizar la Vista Materializada sprint3_gold.mv_daily_sales que vas a crear en el ejercicio anterior. Esto garantiza que el análisis de tendencias sea instantáneo y no consuma recursos de cómputo innecesarios.



Tarea:

Crea una consulta sobre la vista materializada que genere un informe con las siguientes 4 columnas:

- » 1.Fecha: La fecha de venta.
- » 2.Vendes_Avui: El total vendido ese día.
- » 3.Vendes_Ahir: El total vendido el día anterior (usando funciones de ventana).
- » 4.Diff_Percentual: El porcentaje de crecimiento o decrecimiento respecto al día anterior, redondeado a 2 decimales.



Pista Técnica:

Utiliza la función LAG(camp) OVER (ORDER BY data) para leer el valor de la fila anterior sin necesidad de hacer cálculos complejos.

Archivo

Editar

Ejecutar

Herramientas

```

1 SELECT
2   sale_date AS Fecha,
3   total_sales AS Vendes_Avui,
4   LAG(total_sales) OVER (ORDER BY sale_date) AS Vendes_Ahir,
5   ROUND(SAFE_DIVIDE(total_sales - LAG(total_sales) OVER (ORDER BY sale_date),
6     LAG(total_sales) OVER (ORDER BY sale_date)) * 100, 2) AS Diff_Percentual
7 FROM
8   `sprint3_gold.mv_daily_sales`
9 ORDER BY
10  sale_date;
11

```

Esta consulta procesará 816 B cuando se ejecute.

	Información del trabajo	Resultados	Visualización	JSON
Fila	Fecha	Vendes_Avui	Vendes_Ahir	Diff_Percentual
1	2026-04-24	241060.1200000...	null	null
2	2026-04-25	526466.9400000...	241060.1200000...	118.4
3	2026-04-26	520346.5700000...	526466.9400000...	-1.16
4	2026-04-27	512642.0800000...	520346.5700000...	-1.48
5	2026-04-28	504443.0700000...	512642.0800000...	-1.6
6	2026-04-29	521534.1400000...	504443.0700000...	3.39
7	2026-04-30	518289.7900000...	521534.1400000...	-0.62
8	2026-05-01	517955.1700000...	518289.7900000...	-0.06
9	2026-05-02	546212.5000000...	517955.1700000...	5.46

DESARROLLO:

- Utilizamos la Función Ventana **LAG()**, que nos permite acceder al valor de una fila anterior dentro de un conjunto de datos ordenado.
- Esta expresión devuelve el valor de ventas del día anterior (**Vendes_Ahir**) según el orden cronológico definido por **sale_date**.
- Así podemos comparar las ventas del día actual con las del día anterior sin necesidad de hacer subconsultas ni JOINS.
- **LAG(total_sales) OVER (ORDER BY sales_data) AS Vendes_Ahir**.Nos permite comparar el día actual con el día anterior.
- **ROUND(SAFE_DIVIDE(total_sales – LAG (total_sales) OVER (ORDER BY sale_date)) *100, 2) AS Diff _Porcentual**. Calculamos la variación porcentual entre el total de ventas del dia actual y del dia anterior. El resultado con 2 decimales.

Ejercicio 3: Totales Acumulados (Running Totales sobre Vistas)

Escenario:

El Director Financiero (CFO) necesita visualizar la curva de crecimiento anual. No le interesa tanto lo que se vendió hoy, sino cuánto llevamos facturado en total desde el inicio del año hasta hoy (YTD - Year To Date).

🧩 Tarea:

Utilizando la vista materializada mv_daily_sales (para no recalcular agregaciones base), genera un informe con tres columnas:

- » 1.Data.
- » 2.Ventas del Día: Redondeado a 2 decimales.
- » 3.Ventas Acumuladas YTD: Una suma progresiva de las ventas que debe reiniciarse cada 1 de Enero. El resultado final debe estar redondeado a 2 decimales.

💡 Pista Técnica:

Para conseguir el reinicio anual, necesitas dividir la ventana usando PARTITION BY EXTRACT(YEAR FROM data). La estructura completa sería: SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

Archivo	Editar	Ejecutar	Herramientas
1	SELECT		
2	sale_date AS Data,		
3	ROUND(total_sales, 2) AS Ventes_del_Dia,		
4	ROUND(SUM(total_sales) OVER (PARTITION BY EXTRACT(YEAR FROM sale_date)		
5	ORDER BY sale_date		
6	ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW),2) AS Ventes_Acumulades_YTD		
7	FROM `sprint3_gold.mv_daily_sales`		
8	ORDER BY sale_date;		
9			

✅ Esta consulta procesará 816 B cuando se ejecute.

< Información del trabajo		Resultados	Visualización	
Fila	Data ▼	Ventes_del_Dia ▼	Ventes_Acumula...	
1	2026-04-24	241060.12	241060.12	
2	2026-04-25	526466.94	767527.06	
3	2026-04-26	520346.57	1287873.63	
4	2026-04-27	512642.08	1800515.71	
5	2026-04-28	504443.07	2304958.78	
6	2026-04-29	521534.14	2826492.92	
7	2026-04-30	518289.79	3344782.71	
8	2026-05-01	517955.17	3862737.88	

DESARROLLO:

- **SUM(...) OVER(...)** :calculamos la suma acumulada dentro del año.
- **PARTITION BY EXTRACT(YEAR FROM sale_date)**: reinicia el acumulado cada 1 de enero.
- **ORDER BY sale_date**: realizamos el seguimiento en orden cronológico.
- **ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**: incluimos todas las fechas desde el inicio del año hasta la actual.
- **ROUND(..., 2)**: redondeamos el acumulado a 2 decimales.
- **ORDER BY sale_date**: muestra la evolución temporal.
- **CONCLUSIÓN**: Este código nos permite evaluar la evolución del total de ventas diarias dentro del año actual, calculando el acumulado desde el 1 de enero hasta cada fecha mediante la **función ventana SUM(...) OVER (PARTITION BY EXTRACT(YEAR FROM sale_date) ORDER BY sale_date)**.

Ejercicio 4: Fidelización y Valor del Cliente (Filtrado Avanzado)

Escenario:

El equipo de Marketing lanza la campaña "A la tercera va la vencida". Quieren enviar un regalo a los usuarios que hayan completado su tercera transacción. Sin embargo, el presupuesto del regalo no es fijo: depende de

la "calidad" del cliente en sus inicios. Necesitan saber el Ticket Medio de estas 3 primeras compras para segmentar si envían un detalle básico o un regalo premium.

🔗 Tarea:

Genera un listado de los usuarios que han llegado (o superado) su tercera compra, mostrando:

- » 1.Datos de usuario (user_id, nom_complet, email).
- » 2.La fecha y el importe exacto de la 3ª compra.
- » 3.Media de las 3 primeras: La media de gasto de sus transacciones 1, 2 y 3.

💡 Pista Técnica:

- » Debes utilizar la función ROW_NUMBER() para establecer el orden cronológico de las compras.
- » Debes utilizar la cláusula QUALIFY para filtrar.
- » Reto de Optimización: Busca la estrategia de menor coste computacional. Ten en cuenta que calcular medias sobre todo el historial histórico es costoso. Intenta filtrar las filas necesarias antes de realizar la agregación final.

```
1 WITH compres_ordenades AS (  
2   SELECT  
3     user_id,  
4     DATE(timestamp) AS data_compra,  
5     amount,  
6     ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY timestamp) AS num_compra  
7   FROM `sprint3_gold.fact_transactions_optimized`  
8 )  
9 SELECT  
10  u.user_id,  
11  CONCAT(u.name, ' ', u.surname) AS nom_complet,  
12  u.email,  
13  c.data_compra AS data_3a_compra,  
14  c.amount AS import_3a_compra,  
15  ROUND(AVG(c.amount) OVER (PARTITION BY c.user_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW),  
16  2) AS med_3_primeres  
17 FROM compres_ordenades c  
18 JOIN `sprint3_silver.users_combined` u  
19 ON c.user_id = u.user_id  
20 QUALIFY num_compra = 3  
21 ORDER BY med_3_primeres DESC;
```

✅ Esta consulta procesará 2.55 MB cuando se ejecute.

< Información del trabajo		Resultados		Visualización	JSON	Detalles de la ejecución	>
Fila	user_id	nom_complet	email	data_3a_compra	import_3a_compra	med_3_primeres	
1	318	Bnyr Astuw	bnyr.astuw@example...	2026-04-25	615.51	615.51	
2	514	Hxhxec Cbeypjmr	hxhxec.cbeypjmr@ex...	2026-04-29	593.58	593.58	
3	401	Uxhj Ctsfxoqzp	uxhj.ctsfxoqzp@exa...	2026-04-26	567.44	587.87	
4	2289	Qhvdcx Xmjycmgc	qhvdcx.xmjycmgc@e...	2026-04-29	583.42	583.42	
5	479	Qirpsrzg Canoxo...	qirpsrzg.canoxoykp@...	2026-05-03	337.91	562.72	
6	1100	Lxdkva Opfzupna	lxdkva.opfzupna@exa...	2026-05-02	560.61	560.61	
7	187	David Vance	vulputate.velit.eu@pr...	2026-04-26	559.47	559.45	
8	349	Yxxcjk Fnecqbks...	yxxcjk.fnecqbksmt@e...	2026-04-28	489.59	556.6	
9	2596	Sqtagu Veiwksfr	sqtagu.veiwksfr@exa...	2026-05-09	551.33	551.33	
10	1400	Ydkhjc Wilqzhig	ydkhjc.wilqzhig@exa...	2026-05-04	549.03	549.03	

DESARROLLO:

- **compras_ordenadas:** Creamos la tabla temporal con **WITH**.
- **ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY timestamp):** Numera las compras de cada usuario según su orden cronológico. Ejemplo: la primera compra tiene num_compra = 1, la segunda = 2, etc.
 - **ROW_NUMBER():** permite asignar un numero de fila dentro de cada grupo.
 - **OVER():** la numeración se aplica sobre una ventana de datos.
 - **PARTITION BY users_id:** Divide los datos por usuario. Cada usuario empieza su numeración desde el num. 1
 - **ORDER BY timestamp:** Ordena las compras por fecha y hora.
- **ROUND(AVG(c.amount) OVER (PARTITION BY c.user_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS med_3_primeres**
 - **AVG(c.amount):** Calcula la media del importe de las compras.
 - **OVER():** función ventana.
 - **PARTITION BY c.user_id:** Divide los datos por usuario. Cada usuario tiene su propia ventana.
 - **ROWS BETWEEN 2 PRECEDING AND CURRENT ROW:** Selecciona la fila actual y las 2 anteriores (compra 1 = 2 preceding, compra 2 = 1 preceding, compra 3 = current row.
 - **ROUND(...,2):** Redondeo a 2 decimales.

RESULTADO FINAL:

- **1.** Datos de usuario (user_id, nom_complet, email).
- **2.** La fecha y el importe exacto de la 3ª compra.
- **3.** Media de las 3 primeras compras: La media de gasto de sus transacciones 1, 2 y 3.

Nivel 3: Analytics Engineering (Arrays & Automatización)

Escenario:

Marketing necesita urgentemente un Ranking de Productos más Vendidos.

Ejercicio 1: Desanidamiento y Allanamiento de Datos (Unnesting)

Escenario:

Actualmente, en la tabla de transacciones, los productos vendidos están "escondidos" dentro de un Array (una lista de IDs). El equipo de ventas quiere analizar qué productos específicos se venden más, pero no pueden hacerlo si los datos están comprimidos en una lista. Necesitan una Tabla Detallada (Flat Table) donde cada fila represente un producto vendido, aunque esto signifique que el ID de la transacción se repita.

🧩 Tarea:

Crea la tabla dim_transactions_flat desnormalizando la información. Debes "explotar" el array de productos y cruzarlo con el catálogo maestro para obtener los nombres y precios individuales.

⚙️ Pasos Técnicos:

» 1. Utiliza CROSS JOIN UNNEST(product_ids) para transformar el Array en filas individuales.

» 2. Realiza un JOIN con la tabla products para obtener el nombre (name) y precio (price) de cada ítem.

📌 Nota: Ves con cuidado con los tipos de datos al hacer el JOIN (Array de INT64 vs Product_ID STRING).

» 3. No agrupe el resultado. Queremos ver el desglose línea por línea.

☑️ Resultado Esperado (Visualización):

Tu tabla final debe verse "plana", donde una transacción con 3 productos ocupa 3 filas:

transaction_id	timestamp	total_ticket (Global)	product_sku	product_name	product_price
TXN1001	2025-12-08	155.97	SKU_A	Portàtil Bàsic	99.99
TXN1001	2025-12-08	155.97	SKU_B	Ratolí	12.99
TXN1001	2025-12-08	155.97	SKU_C	Funda	29.99
TXN1002	2025-12-07	45.50	SKU_D	Teclat	45.50



Entrega:

- » Una captura de la Vista Previa de tu tabla donde se vea el DDL ejecutada correctamente.
- » Una captura de pantalla donde se vea el esquema de la nueva tabla.

```

Archivo  Editar  Ejecutar  Herramientas
1 CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS
2 SELECT
3     t.transaction_id,
4     t.timestamp,
5     t.amount AS total_ticket,
6     product_id AS product_sku,
7     p.name AS product_name,
8     p.price AS product_price
9 FROM `sprint3_silver.transactions_clean` t
10 CROSS JOIN UNNEST(t.product_ids) AS product_id
11 LEFT JOIN `sprint3_silver.products_clean` p
12 ON product_id = p.product_id;
13

```

✓ Esta consulta procesará 7.09 MB cuando se ejecute.

Filtro Escribir el nombre o valor de la propiedad

☐	Nombre del campo	Tipo	Modo
☐	transaction_id	STRING	NULLABLE
☐	timestamp	TIMESTAMP	NULLABLE
☐	total_ticket	FLOAT	NULLABLE
☐	product_sku	INTEGER	NULLABLE
☐	product_name	STRING	NULLABLE
☐	product_price	FLOAT	NULLABLE

Resultados de la consulta

Chat

View all jobs

Guardar los resultados ▾

Abrir en ▾

<

Información del trabajo

Resultados

Visualización

JSON

Detalles de la ejecución

Gráfico

	transaction_id ▾	timestamp ▾	total_ticket ▾	product_sku ▾	product_name ▾	product_price ▾
1	3A4E741A-1653-4F5F-91B1-ED...	2022-05-03 02:03:29 UTC	177.67	14	Direwolf	147.53
2	51C692E3-4D69-4DAD-B4D6-D6...	2022-06-22 07:26:23 UTC	278.77	14	Direwolf	147.53
3	5975B711-ACE5-4ACE-A34C-90...	2020-11-30 15:29:23 UTC	703.68	14	Direwolf	147.53
4	DD903230-8925-45FF-B9D3-1C...	2022-06-06 17:46:57 UTC	411.22	14	Direwolf	147.53
5	B5382D88-33E1-4085-8951-EE...	2018-02-01 02:31:02 UTC	319.1	14	Direwolf	147.53
6	5AE2402A-3EE7-4C5A-B924-A9...	2021-12-05 09:29:58 UTC	301.37	14	Direwolf	147.53
7	9CA2845C-D567-4974-AC0B-E3...	2020-10-21 02:53:20 UTC	219.42	14	Direwolf	147.53

DESARROLLO:

- Hemos creado la tabla `dim_transactions_flat` a partir de los datos de transacciones y del catálogo de productos.
- Hemos desnormalizado la información explotando la columna `product_ids` mediante `CROSS JOIN UNNEST`, generando una fila por cada producto incluido en una transacción.
- Hemos realizado un `JOIN` con la tabla de productos para incorporar el nombre y el precio de cada artículo.
- **Resultado final:** una tabla completamente plana que muestra, línea por línea, cada combinación de transacción y producto, junto con el importe total del ticket y la información detallada del ítem.

Ejercicio 2: El Ranking de Ventas (Agregación Simple)

Escenario:

Ahora que has preparado la infraestructura y dispones de la mesa `dim_transactions_flat` (donde cada fila equivale a un producto vendido), generar informes analíticos es trivial. Ya no necesitas funciones complejas de desanidamiento para responder preguntas básicas de negocio.

 Tarea:

Genera el Top 5 de productos más vendidos en la historia de la compañía.

 Método:

Consulta directamente tu nueva mesa desnormalizada (`dim_transactions_flat`).

» Al ser la tabla plana, simplemente necesitas una agrupación estándar (`GROUP BY`) por el nombre del producto y una cuenta (`COUNT`) de las filas.

🔗 Entrega:

- » El código SQL que genera el ranking.
- » Una captura del resultado con los 5 productos ganadores y cuántas unidades se han vendido de cada uno.

```
Archivo  Editar  Ejecutar  Herramientas

1  SELECT
2      product_name,
3      COUNT(*) AS unidades_vendidas
4  FROM `sprint3_gold.dim_transactions_flat`
5  GROUP BY product_name
6  ORDER BY unidades_vendidas DESC
7  LIMIT 5;
8
```

✅ Esta consulta procesará 4.06 MB cuando se ejecute.

< Información del trabajo		Resultados	Visualización
Fila	product_name ▼	unidades_vendidas ▼	
1	Winterfell	10071	
2	the duel	10059	
3	dooku solo	10022	
4	Karstark Dorne	7694	
5	skywalker ewok	7607	

DESARROLLO:

- Utiliza la tabla desnormalizada dim_transactions_flat, donde cada fila representa un producto vendido.
- Agrupamos por product_name y contamos cuántas veces aparece cada producto.
- Ordenamos por volumen de ventas y seleccionamos los cinco artículos más vendidos.

Ejercicio 3: Automatización del Pipeline y Visualización

Escenario:

El informe estático del ejercicio anterior ha gustado mucho, pero el Director de Marketing necesita profesionalizarlo para la toma de decisiones diaria. Tiene dos requerimientos urgentes:

» 1. Lógica Centralizada: Quieren ver el precio con IVA (21%) desglosado. Sin embargo, como el impuesto puede cambiar por país o año, te prohíben poner el cálculo $\text{precio} * 1,21$ directamente en el código ("Hardcoded"). Debes utilizar una función reusable.

» 2. Frescura de Datos: El tablero debe estar actualizado automáticamente cada mañana a las 07:00 AM, antes de la reunión diaria de ventas.

🔧 Tarea:

Debes evolucionar tu tabla `dim_transactions_flat` para incluir el cálculo de impuestos y automatizar su regeneración.

⚙️ Pasos Técnicos:

» 1. User Defined Functions (UDF): Crea una función SQL persistente llamada `calculate_tax(amount)` que reciba un valor numérico y devuelva el resultado aplicando el 21% de impuesto.

» 2. Integración y Orquestación:

Modifica el código de creación de la tabla (del Ejercicio 1) para que utilice tu nueva función `calculate_tax` y genere una columna nueva: `product_price_tax_inc`.

Configura una Scheduled Query en BigQuery para que esta sentencia (CTAS) se ejecute automáticamente cada día a las 07:00 AM.

» 3. Visualización (BI): Conecta Looker Studio a tu mesa `dim_transactions_flat` y crea un Dashboard: "Monitor de Rendimiento de Ventas"

📎 Entrega:

» El código SQL de la UDF.

	Archivo	Editar	Ejecutar	Herramientas
1	CREATE OR REPLACE FUNCTION `sprint3_gold.calculate_tax`(amount FLOAT64)			
2	RETURNS FLOAT64			
3	AS (amount * 1.21			
4	);			
5				

✅ Esta consulta procesará 0 B cuando se ejecute.

» El código SQL actualizado de la creación de la tabla.

Archivo Editar Ejecutar Herramientas

```
1 CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS
2 SELECT
3     t.transaction_id,
4     t.timestamp,
5     t.amount AS total_ticket,
6     product_id AS product_sku,
7     p.name AS product_name,
8     p.price AS product_unit_price,
9     `sprint3_gold.calculate_tax`(p.price) AS product_price_tax_inc
10 FROM `sprint3_silver.transactions_clean` t
11 CROSS JOIN UNNEST(t.product_ids) AS product_id
12 LEFT JOIN `sprint3_silver.products_clean` p
13 ON product_id = p.product_id;
14
```

✓ Esta consulta procesará 7.09 MB cuando se ejecute.

» Una captura de pantalla demostrando que la Scheduled Query está programada.

Exportar como consulta programada

Frecuencia de repetición * ▼
Días

A la(s) * 07:00 UTC

☐ Comenzar ahora ☒ Comenzar a la hora definida

Fecha de inicio y hora de ejecución * 17/6/26, 7:00 a.m. 000 📅

☒ Sin hora de finalización programada
☐ Programar una hora de finalización

Fecha de finalización 000 📅

i Programado para ejecutarse todos los días a las 07:00 UTC a partir del 17 de junio de 2026 a las 7:00:00 a.m. 000.

Destino de los resultados de la consulta

☒ Establecer una tabla de destino para los resultados de la consulta

Conjunto de datos * 📌 sprint3-analytics-albert.sprint3_gold

» Un enlace del gráfico a Looker Studio.

Informe sin título							Restablecer	Compartir	Ver			
Archivo Editar Vista Insertar Página Organizar Recurso Ayuda												
Fuentes de datos												X CERRAR
Nombre	Tipo de conector	Tipo	Se usa en el informe	Estado	Acciones		Alias					
dim_transactions_flat	BigQuery	Insertada	5 gráficos, 0 variables	Activa	EDITAR	DUPLICAR	ELIMINAR	HACER REUTILIZABLE	ds0			

-  Propuesta de Dashboard:
- "Monitor de Rendimiento de Ventas"
- Configura una página con 3 secciones clave:
- » 1. Los "Big Numbers" (KPIs Generales):
- En la parte superior, coloca tres tarjetas de resultados (Scorecards) para dar contexto inmediato.

- KPI 1: Ingresos Totales (Con IVA)
- Métrica: SUM(product_price_tax_inc)
- Por qué: Muestra el volumen real de dinero que entra.
- Transacciones Únicas (Pedidos)
- Métrica: COUNT_DISTINCT(transaction_id)
- Transacciones Únicas (Pedidos)
- Campo Calculado: Crea un campo en Looker:
SUM(product_price_tax_inc) /
COUNT_DISTINCT(transaction_id)
- » 2. Evolución Temporal (Gráfico de Series Temporales)
- Bajo los KPIs, un gráfico de línea que ocupe todo lo ancho.
- Absorción: timestamp (Desglosado por Fecha).
- Métrica: SUM(product_price_tax_inc).
- Línea de Referencia (Opcional): Añade una media constante o una línea de tendencia.
- Valor: Permite a Marketing ver si las ventas suben o bajan día a día, o si hubo un pico para alguna campaña específica.
- » El "Top Productos" Enriquecido (Tabla con Mapa de Calor)
- Olvídate del gráfico de barras simple. Utiliza una Mesa con Barras incrustadas o Mapa de Calor.
- Absorción: product_name.
- Métrica 1: COUNT(*) → Renombrarlo a "Unidades Vendidas".
- Métrica 2: SUM(product_price_tax_inc) → Renombrarlo a "Ingresos Totales".
- Métrica 3: SUM(product_price_tax_inc) - SUM(product_unit_price) → Renombrarlo a "IVA Recaudado" (Esto demuestra el valor de tu UDF).
- Estilo: En la configuración de estilo de la tabla, activa la opción "Muestra número como: Barra" o "Mapa de calor" para la columna de Ingresos.

Ingresos Totals (amb IVA)

31,34 M €

Número Transacciones Únicas

100.000

Ingreso medio por transacción

313,41 €

Informe General

Nombre Prod...	Unidades Vendidas ▾	Ingresos Totales	IVA Recaudado
1.. Winterfell	10.071	1.055.856,15	183.247,76
2.. the duel	10.059	1.274.589,23	221.209,7
3.. dooku solo	10.022	695.043,83	120.627,44
4.. Karstark Dorne	7.694	852.007	147.868,98
5.. skywalker ewok	7.607	1.539.019,97	267.102,64
6.. Karstark warden	7.563	1.247.647,83	216.533,92
7.. Direwolf Stannis	7.429	846.455,8	146.905,55
8.. duel Direwolf	5.167	867.987,33	150.642,43
9.. duel warden	5.166	891.406,69	154.706,95
1.. Lannister	5.155	565.212,9	98.094,8
1... north	5.134	1.055.569,97	183.198,09

1 - 70 / 70<>

Evolución Temporal de los Ingresos Totales (con IVA)

